Search in Constraint Programming (CP)

Overview and Context

While Constraint Propagation is a powerful deduction mechanism, it is rarely sufficient to completely solve complex combinatorial problems on its own. When propagation reaches a fixed-point and cannot deduce any further domain reductions, the solver must make an educated guess to proceed. This process of guessing, exploring, and backtracking is known as **Search**.

This lesson explores the architecture of the **Constraint Solver**, which effectively merges Backtracking Tree Search (BTS) with Constraint Propagation. The primary learning objectives are to understand the mechanics of branching, the critical importance of interleaving search with propagation to prune the search tree, and the strategic use of **Search Heuristics**—particularly the Fail-First principle—to navigate exponential spaces. Furthermore, it introduces advanced strategies like Restarts to mitigate "Heavy Tail" behavior and the Branch & Bound algorithm for solving Constraint Optimization Problems (COPs).

1. Backtracking Tree Search (BTS) and Branching

Explanation: At its foundation, search in CP operates as a systematic **Backtracking Tree Search (BTS)** algorithm that enumerates possible variable-value combinations to find a solution. The search space is represented as a tree where each **node** represents a variable X_i, and each **branch** represents a specific decision made on that variable. A decision usually consists of posting a unary constraint on X_i to partition its domain. If the search encounters a dead-end (an infeasible state), it performs **chronological backtracking**, retracting the most recently posted decision to try an alternative path.

- **d-way branching:** The solver creates a separate branch for every single value v_j in the domain $D(X_i)$, formulating decisions as $X_i = v_j$.
- **2-way branching:** The solver splits the domain into two logical paths, typically $X_i = v$ and $X_i \neq v$, or checking if X_i belongs to a specific subset ($X_i \in S$ and $X_i \notin S$).
- **k-way branching (Domain Partitioning):** The domain $D(X_i)$ is split into k disjoint subsets S_j, creating a branch for each $X_i \in S_j$.

Examples & Applications

Pure BTS Complexity: If a problem has n variables, each with a domain size of d, a pure BTS algorithm (without propagation) explores the variables sequentially, generating a tree with a worst-case time complexity of $\mathcal{O}(d^n)$. Because it only checks constraints *after* all variables involved are instantiated, it scales exponentially and is completely impractical for real-world applications.

2. Interleaving Search and Propagation

Explanation: A true modern Constraint Solver does not merely use pure BTS; it heavily relies on **interleaved propagation**. At every single node in the search tree, after a branching decision is made, the solver immediately triggers its propagation engine. This active logical inference removes inconsistent values from the domains of *future* (unexplored) variables before the search even reaches them. This synergy drastically shrinks the search space, transforming intractable search trees into solvable problems.

- **Pre-search vs. In-search:** The solver propagates all constraints completely before search begins, and then only propagates the necessary (woken) constraints dynamically during the search process.
- **Early Dead-End Detection:** By shrinking future domains, propagation forces variables to empty domains much higher up in the search tree, triggering immediate backtracking and preventing the solver from exploring massive, fundamentally flawed sub-trees.

Examples & Applications

The 4-Queens Tree Reduction: In the classic 4-Queens problem, a standard search explores numerous invalid partial placements before realizing a constraint is violated. By applying BTS combined with Arc Consistency (AC) propagation, placing the first queen immediately eliminates attacked squares from the domains of the remaining queens. As shown in the visual models, this reduces the search tree from a massive structure to just a handful of nodes, successfully identifying solutions with minimal branching.

3. Search Heuristics and the Fail-First Principle

Explanation: In a search tree, the order in which we choose variables to instantiate (**Variable Ordering Heuristics**, VOH) and the order we try their values dictate how fast we find a solution—or how fast we prove there isn't one. Because CP rarely guarantees feasibility, we must assume we will make mistakes and get trapped in infeasible sub-problems. To handle this, CP relies on the **Fail-First (FF) principle**: *Try first where you are most likely to fail.* If a sub-tree is doomed, we want to prove it as quickly as possible so we can backtrack. Conversely, for values, we choose the one *most likely* to succeed.

- **Minimum Domain:** A dynamic VOH that chooses the variable X_i with the smallest current domain size $|D(X_i)|$. This mathematically minimizes the immediate branching factor of the search tree.
- **Most Constrained (Max Degree):** Chooses the variable involved in the highest number of constraints, aiming to maximize the cascading impact of constraint propagation.
- **domWdeg (Domain over Weighted Degree):** An advanced heuristic that tracks constraint failures. Each constraint c starts with a weight of

1, which increments every time c causes a failure. The solver selects the variable X_i that minimizes the ratio: $\frac{|D(X_i)|}{\sum_{c \text{ s.t. } X_i \in X(c)} w(c)}$.

Examples & Applications

Fail-First Logic (Minimum Domain): Consider three variables: $X_1 \in \{0, 1, 2, 3\}$, $X_2 \in \{0, 1, 2\}$, and $X_3 \in \{0, 1\}$. * If we branch on X_1 first, a failure deep in the tree means we must backtrack and explore 4 massive branches. * If we branch on X_3 first (Minimum Domain), a failure only requires exploring 2 branches. The "Fail-First" approach forces the solver to tackle the narrowest, most restricted parts of the problem immediately, minimizing the size of the exponential sub-trees it might have to explore.

4. Heavy Tail Behaviour and Restart Strategies

Explanation: When solving a set of CSP instances, researchers frequently observe **Heavy Tail Behaviour**: the solver easily solves most instances, but gets stuck for an exceptionally long time on a few "hard" ones. This isn't necessarily a flaw in the instance itself, but rather an unlucky combination of the problem and the specific heuristic order. If the heuristic makes a "bad" mistake at the very top of the tree, the solver gets trapped evaluating an exponentially large, completely infeasible sub-tree. To break out of these traps, solvers employ **Randomization** coupled with **Restarts**.

- **Randomization:** Adding a random factor to heuristic ties or choices so the solver never explores the exact same search tree twice.
- **Restarting:** Halting the search after a specific resource limit L (like a node count limit) is reached, and starting over from the root node. Crucially, learned information (like `domWdeg` failure weights) is preserved across restarts to guide the next pass more intelligently.
- **Restart Sequences:**
  - *Geometric:* Limits grow by a factor α (e.g., $L, \alpha L, \alpha^2 L \ldots$).
  - *Luby:* A mathematically robust sequence of limits based on powers of 2 (e.g., $1, 1, 2, 1, 1, 2, 4 \ldots$) multiplied by L.

Examples & Applications

Rescuing domWdeg with Restarts: The `domWdeg` heuristic is highly synergistic with restarts. In the first run, the solver might get trapped in a heavy tail, but it rapidly accumulates high "weights" on the constraints causing the trap. Upon restarting, the `domWdeg` formula utilizes these preserved fail counts to immediately branch on the problematic variables, avoiding the trap entirely in the second run.

5. Constraint Optimization Problems (COPs) and Branch & Bound

Explanation: Many real-world applications require finding not just *a* valid solution, but the *best* valid solution (e.g., minimum cost). This is formalized as a **Constraint Optimization Problem (COP)**: $< X, D, C, f >$, where f is the objective function to minimize (or maximize). COPs are typically solved using the **Branch & Bound** algorithm. Instead of stopping when a feasible solution is found, the solver temporarily saves it, injects a new constraint strictly bounding the objective, and forces the search to continue looking for a strictly better solution.

- **Algorithm Steps:**
  1. Find a feasible solution S_1 with objective value $f(S_1)$.
  2. Post a new bounding constraint dynamically: $f(X) < f(S_1)$ (assuming a minimization problem).
  3. Backtrack and continue the search tree. Any branch that mathematically cannot beat the bound is pruned.
  4. Repeat until the search space is exhausted (proven infeasible). The last solution found is globally optimal.

Examples & Applications

Optimal Map Colouring: We want to colour a map using the minimum number of colours. The objective is $f(X) = \max(X_i)$ for $X_i \in \{1 \dots n\}$. * The solver finds a valid colouring using 4 colours ($f = 4$). * It immediately posts the bounding constraint: $\max(X_i) < 4$. * The solver explores the remaining tree. If it finds a 3-colour solution, it updates the bound to $\max(X_i) < 3$. * When the solver exhausts the search space and fails to find a 2-colour solution, the 3-colour solution is proven optimal.

6. Large Neighbourhood Search (LNS)

Explanation: While CP is excellent at strict feasibility and constraint propagation, it often struggles to scale on massive optimization problems because its bounding logic is generally weaker than Operations Research approaches. Heuristic Search (HS) scales well but struggles with strict logic. **Large Neighbourhood Search (LNS)** is a hybrid meta-heuristic that combines their strengths.

- **The LNS Loop:**
  1. Use CP or a greedy heuristic to find an initial, sub-optimal solution s.
  2. Select a subset of variables (a "fragment") and **fix** them to their current values in s.
  3. **Relax** (unassign) the remaining variables.
  4. Use the CP solver with heavy propagation to exhaustively search for the optimal assignment for *only* the relaxed variables, treating it as a

highly constrained, small sub-problem.
5. Update the solution and repeat.

Synthesis and Conclusions

Search is the necessary counterpart to propagation in a CP solver. By interleaving Backtracking Tree Search with continuous Constraint Propagation, solvers achieve exponential reductions in the search space. Because the architecture of a search tree heavily dictates performance, selecting dynamic Variable Ordering Heuristics via the Fail-First principle (like Minimum Domain or `domWdeg`) is critical to rapidly proving dead-ends. When these heuristics fail and cause "Heavy Tails", randomized Restarts allow the solver to mathematically escape and leverage learned constraint weights. Finally, for optimization, the CP engine is adapted using the Branch & Bound algorithm to continually tighten bounds, or it is embedded into hybrid techniques like Large Neighbourhood Search (LNS) to achieve high scalability on massive industrial problems.

Essential Glossary

- **Backtracking Tree Search (BTS):** A systematic algorithm that recursively explores variable assignments, backtracking upon reaching dead-ends.
- **Fail-First (FF) Principle:** A heuristic philosophy directing the solver to tackle the most restricted parts of the problem first to quickly identify infeasible search paths.
- **Variable Ordering Heuristic (VOH):** The logic used by the solver to determine which variable X_i to branch on next during the search.
- **domWdeg:** A dynamic VOH that prioritizes variables with small domains that are heavily involved in constraints which have historically caused search failures.
- **Heavy Tail Behaviour:** A phenomenon where certain problem instances take an extraordinarily long time to solve due to poor early branching decisions trapping the solver in massive sub-trees.
- **Branch & Bound:** An algorithm used for Constraint Optimization Problems (COPs) that dynamically tightens the objective bound every time a new solution is found, ensuring continuous improvement until optimality is proven.
- **Large Neighbourhood Search (LNS):** A hybrid solving method that iteratively improves a solution by freezing a fragment of variables and using CP to optimally re-solve the remaining relaxed variables.